

# SI26\_P13: Festival Check-In Join Explorer

*AI-Assisted Coding Interview Problem*

---

## Problem Statement

Build a compact Festival Check-In Join Explorer for a college festival team. It should compare the festival registration list with the entry-gate check-in list so volunteers can see who checked in, who registered but has not arrived, and which check-ins do not have a registration.

Use one attractive primary screen or report with two small input tables, a comparison action, a result table, status counts, and sample/reset controls. The tables may be editable directly or through one focused row form. You may use SQL, in-memory data structures, a spreadsheet or notebook, a browser, desktop or mobile tools, or a CLI that produces a clear visual or tabular report. No backend, database, account, network service, or file upload is required.

Load this built-in example:

### Registrations

| Registration ID | Student    | Festival Activity |
|-----------------|------------|-------------------|
| F101            | Ananya Rao | Robo Race         |
| F102            | Dev Menon  | Music Night       |
| F103            | Isha Patel | Open Mic          |
| F104            | Farhan Ali | Poster Sprint     |

### Check-ins

| Registration ID | Gate       | Check-in Time |
|-----------------|------------|---------------|
| F101            | North Gate | 09:05         |
| F103            | East Gate  | 09:12         |
| F999            | North Gate | 09:18         |

Treat registrations as the left side of a relational comparison keyed by registration ID. The built-in result has five rows: F101 and F103 are MATCHED; F102 and F104 are REGISTERED\_ONLY; and F999 is an ORPHAN\_CHECK\_IN. The summary is therefore 2 matched, 2 registered only, and 1 orphan check-in.

## Contracts

- Trim surrounding spaces from registration IDs before comparison. IDs are otherwise case-sensitive and must be non-empty. An empty ID reports `INVALID_ID` with the table and row and clears any earlier result and counts.
- A registration ID must be unique within each input table. If either table contains a duplicate after trimming, report `DUPLICATE_KEY` with the table name and duplicate ID, and show no result rows or status counts from an earlier comparison.
- For each registration, produce `MATCHED` when exactly one check-in has the same ID; otherwise produce `REGISTERED_ONLY`. Then append every check-in whose ID does not occur in registrations as `ORPHAN_CHECK_IN`.
- Preserve registration-table order for the first group of result rows, followed by orphan check-ins in check-in-table order.
- A result row shows the ID, status, and all available registration and check-in values. Use a visible placeholder such as – for fields that do not exist on one side.
- Either input table may be empty. With no check-ins, every registration is `REGISTERED_ONLY`; with no registrations, every check-in is `ORPHAN_CHECK_IN`.

### Acceptance criteria

- **Required:** Load the built-in pair of tables in one action and produce five result rows in the contracted order.
- **Required:** Show `MATCHED` for F101 and F103, `REGISTERED_ONLY` for F102 and F104, and `ORPHAN_CHECK_IN` for F999, with summary counts 2, 2, and 1 respectively.
- **Required:** Clear the check-in table and compare again; show four `REGISTERED_ONLY` rows, zero `MATCHED` rows, and zero `ORPHAN_CHECK_IN` rows.
- **Required:** Add a second F101 row to the check-in table, report `DUPLICATE_KEY` naming check-ins and F101, and clear any previous result and summary.
- **Required:** Keep both input tables, the result table, status counts, validation message, and sample/reset actions synchronized, and include focused checks for the built-in, empty-check-in, and duplicate-key cases.
- **Optional:** Add a result-status filter while keeping the summary counts based on the complete comparison.

Use AI coding assistants. Before implementation, create a short plan with 3–5 ordered steps and useful checkpoints. Be prepared to present that plan, explain any changes you made to it, share relevant prompts, summarize your design, and show test evidence such as tests, screenshots, or output samples.

## How You'll Be Evaluated

- **Planning and Solution Presentation:** Present your 3–5-step implementation plan, explain how the work followed or changed that plan, and demonstrate the working solution with clear explanations

- **AI Prompting Strategy:** Show the prompts you used to translate this problem statement into technical specifications for AI assistants
- **Design Constraints and Technology Choices:** Explain the constraints you provided to AI regarding design patterns, technology stack, and architectural decisions
- **AI-Influenced Decision Making:** Discuss trade-offs, assumptions, and how AI recommendations influenced your choices for components, data structures, and implementation approaches
- **Testing and Validation:** Demonstrate how you tested the application covering both typical usage scenarios and edge cases
- **Live Modification Capability:** Be prepared to implement one small modification, and possibly a second if time permits, using AI assistance; keep your development environment ready for focused changes and verification